FACTUAL KNOWLEDGE → LML/NSL INTEGRATION

Piping 343 Fact Types Through Universal Compiler

Input: 343 factual knowledge types from EFUE collapse
Pipeline: UIC → UUIS → LML + NSL
Output: Executable knowledge laws + fact repair moves

UIC: UNIVERSAL IDENTITY COMPILER

Input Artifact:

343 Fact Type Classes (8 dimensions × 9 levels each)

UIC Step 1: Extract Primitives

State Variables (x):

- Asserted facts: Set of (subject, relation, object, context) tuples
- Fact types: [K1.x, K2.x, K3.x, K4.x, K5.x, K6.x, K7.x, K8.x] for each fact
- Verification status: {verified, unverified, contradicted}
- Knowledge graph: Nodes (entities) + Edges (relations)

Invariants (I):

1. **Fact Type Consistency:** Every fact MUST map to exactly one of 343 types
    - "MUST specify relation type (K1.x)"
    - "MUST specify domain (K2.x)"
    - "MUST specify all 8 dimensions"
2. **Non-Contradiction:** MUST NOT assert contradictory facts
    - "CANNOT assert 'A is B' AND 'A is not B'"
    - "CANNOT assert 'X happened at T1' AND 'X happened at T2' (if X is unique event)"
3. **Verification Requirement:** Facts MUST have verification path matching K4.x

- "IF K4.1 (mathematical) THEN must have proof"
- "IF K4.3 (experimental) THEN must have experimental evidence"
- "IF K4.4 (historical) THEN must have historical record"

4. **Derivation Integrity:** Derived facts MUST follow from premises
   - "IF K8.3 (one-step derivation) THEN must cite source facts"
   - "IF derived THEN derivation must be valid"
5. **Stability Consistency:** Claimed stability MUST match actual change rate
   - "IF K5.0 (eternal) THEN fact CANNOT change"
   - "IF K5.8 (momentary) THEN fact expected to change constantly"
6. **Scope Validity:** Scope claims MUST match actual applicability
   - "IF K6.0 (universal) THEN must apply everywhere"
   - "IF K6.3 (national) THEN cannot claim applies globally"
7. **Domain Coherence:** Subject and object MUST belong to stated domain
   - "IF K2.4 (mathematical) THEN subject/object must be mathematical entities"

Disturbances (Δ):

- Fact contradiction: New fact contradicts existing
- Verification failure: Cannot verify claimed fact
- Derivation error: Invalid inference
- Staleness: Time-dependent fact becomes outdated
- Scope creep: Fact applied beyond valid scope
- Domain confusion: Entities in wrong domain
- Type mismatch: Fact doesn't fit claimed type

Couplings (A/B):

- Verification type ↔ Derivability (mathematical must be derivable or axiomatic)
- Domain ↔ Relation type (some relations only in some domains)
- Stability ↔ Verification (eternal facts need eternal verification)

- Scope ↔ Domain (universal scope requires domain universality)

UIC Output (PER):

```
PER_FACTUAL_KNOWLEDGE = {
  x: {
    facts: [(subject, relation, object, context, fact_type), ...],
    verification_status: {fact_id → status},
    knowledge_graph: {nodes, edges}
  },

  I: [
    "Every fact MUST map to one of 343 types",
    "MUST NOT assert contradictory facts",
    "Facts MUST have verification path matching type",
    "Derived facts MUST follow from valid premises",
    "Stability claims MUST match change rate",
    "Scope claims MUST match applicability",
    "Domain coherence MUST be maintained"
  ],

  Δ: [
    "Contradiction",
    "Verification failure",
    "Derivation error",
    "Staleness",
    "Scope creep",
    "Domain confusion",
    "Type mismatch"
  ],

  A_B: {
    couplings: [
      "Verification ↔ Derivability",
      "Domain ↔ Relation",
      "Stability ↔ Verification",
      "Scope ↔ Domain"
    ]
  },

  HOLES: [
    "How to detect contradictions? Need fact comparator",
    "How to verify verification? Need verification validator",
    "How to check derivation validity? Need logic checker",
    "How to detect staleness? Need temporal monitor",
    "What to do when contradiction found? Need resolution strategy"
```

```
  ]
}
```

UUIS: UNIVERSAL SNAP COMPILER

Block 1: IDENTITY KERNEL

```
CORE KNOWLEDGE INVARIANTS:

1. Fact Type Completeness Law
   "Every asserted fact MUST be classified into one of 343 types"

2. Non-Contradiction Law
   "Knowledge base MUST NOT contain contradictory facts"

3. Verification Integrity Law
   "Each fact MUST have verification matching its K4.x type"

4. Derivation Validity Law
   "Derived facts (K8.3-K8.7) MUST follow from valid premises"

5. Dimensional Coherence Law
   "All 8 dimensions MUST be internally consistent for each fact"

HOLES:
- Need canonical fact representation
- Need contradiction detection algorithm
- Need verification validator
```

Block 2: LEGALITY REGION

```
FORBIDDEN FACT PATTERNS:

CONTRADICTIONS:
- (A, IS-A, B) ∧ (A, IS-A, ¬B) where B and ¬B are incompatible
- (X, HAPPENED-AT, T1) ∧ (X, HAPPENED-AT, T2) where X is unique event
- (X, EQUALS, Y) ∧ (X, EQUALS, Z) where Y ≠ Z
- (X, LOCATED-AT, L1) ∧ (X, LOCATED-AT, L2) where L1 ≠ L2 (simultaneously)

TYPE MISMATCHES:
- K4.1 (mathematical) ∧ K5.8 (momentary) - impossible
- K4.0 (definitional) ∧ K8.8 (empirical) - contradictory
```

```
- K6.0 (universal) ∧ K6.3 (national) - inconsistent

INVALID COMBINATIONS:
- K1.4 (temporal) ∧ K2.4 (mathematical) - rare, suspicious
- K5.0 (eternal) ∧ K5.8 (momentary) - contradiction

ADMISSIBLE:
- All 343 core fact types
- Proper dimensional alignments
- Verified claims

ENFORCEMENT:
- Pre-check before fact insertion
- Periodic consistency validation
- Automatic contradiction detection

HOLES:
- Need complete forbidden pattern catalog
- Need efficient contradiction checker
```

Block 3: DETECTION STACK

```
FACT INTEGRITY DETECTORS:

Detector 1: Relation Type Classifier
Input: (subject, relation, object)
Output: K1.x classification
Signals: Relation verb, semantic role

Detector 2: Domain Classifier
Input: subject, object entities
Output: K2.x classification
Signals: Entity types, ontology

Detector 3: Granularity Analyzer
Input: Fact structure
Output: K3.x classification
Signals: Complexity, compound structure

Detector 4: Verification Type Detector
Input: Fact + provenance
Output: K4.x classification
Signals: Source type, evidence available

Detector 5: Stability Tracker
Input: Fact + change history
```

```
Output: K5.x classification
Signals: Historical change rate

Detector 6: Scope Analyzer
Input: Fact + context
Output: K6.x classification
Signals: Applicability range

Detector 7: Abstraction Level Detector
Input: Fact content
Output: K7.x classification
Signals: Concreteness, theoretical depth

Detector 8: Derivability Checker
Input: Fact + knowledge base
Output: K8.x classification
Signals: Can be derived vs must be asserted

Detector 9: Contradiction Detector
Input: New fact + existing knowledge base
Output: Contradictions found
Signals: Logical inconsistency, type clash

Detector 10: Staleness Monitor
Input: Fact + timestamp + K5.x stability
Output: Freshness status
Signals: Time elapsed vs expected lifetime

HOLES:
- Need trained classifiers for each dimension
- Need fast contradiction checker (NP-hard in general)
- Need temporal tracking system
```

Block 4: CONTAINMENT TOPOLOGY

```
KNOWLEDGE ORGANIZATION MODULES:

Module A: Core Facts (Highest Authority)
- K4.0 (definitional)
- K4.1 (mathematical)
- K5.0 (eternal)
- Cannot be contradicted by lower modules

Module B: Established Knowledge
- K4.3 (experimental)
- K4.4 (historical)
```

```
- K5.1-K5.3 (stable)
- High confidence

Module C: Current/Consensus
- K4.5 (consensus)
- K4.7 (statistical)
- K5.4-K5.6 (generational to yearly)
- Medium confidence

Module D: Derived/Inferred
- K8.3-K8.7 (derived)
- Depends on premises
- Confidence from source facts

Module E: Temporary/Contextual
- K5.7-K5.8 (daily to momentary)
- K6.7-K6.8 (contextual/hypothetical)
- Low persistence

Coupling Rules:
- Core → Established (strong trust)
- Established → Current (moderate trust)
- Current → Derived (validation needed)
- Temporary ← isolated (no coupling)

Firebreaks:
- Module A contradictions → halt system
- Module B contradictions → flag for review
- Module C contradictions → update mechanism
- Module D contradictions → check derivation

HOLES:
- Need module authority hierarchy
- Need contradiction resolution protocol
```

Block 5: CORRECTION STACK

```
KNOWLEDGE REPAIR MOVES:

Repair 1: Type Classification Correction
IF fact type undetermined or wrong
THEN:
  - Re-analyze using all 8 detectors
  - Assign correct type signature
  - Update storage location
```

```
Repair 2: Contradiction Resolution
IF contradiction detected
THEN:
  - Identify conflicting facts
  - Compare authority (module hierarchy)
  - Options:
    a) Keep higher authority, delete lower
    b) Flag both for human review
    c) Create context-dependent versions
  - Log resolution

Repair 3: Verification Restoration
IF fact lacks required verification
THEN:
  - Check if verification exists elsewhere
  - Attempt to derive from other facts
  - If impossible → downgrade confidence or delete
  - Log verification gap

Repair 4: Derivation Repair
IF derived fact's premises changed
THEN:
  - Re-derive from current premises
  - If derivation fails → delete derived fact
  - If result changed → update derived fact
  - Log derivation update

Repair 5: Staleness Refresh
IF time-dependent fact expired
THEN:
  - Mark as stale
  - Attempt to fetch updated value
  - If updated → replace
  - If unavailable → flag as unknown
  - Log staleness event

Repair 6: Scope Correction
IF fact applied beyond valid scope
THEN:
  - Restrict to valid scope
  - Create scope-qualified version
  - Prevent over-generalization
  - Log scope violation

Repair 7: Domain Reclassification
IF entity assigned to wrong domain
THEN:
```

```
  - Reclassify to correct domain
  - Update all related facts
  - Check for domain-specific rules
  - Propagate changes

HOLES:
- Need authority comparison algorithm
- Need automated derivation system
- Need freshness update mechanism
```

Block 6: RESERVED CAPACITY

```
KNOWLEDGE BASE BUDGETS:

Budget 1: Contradiction Tolerance
- 0 contradictions allowed in Module A (Core)
- <0.01% in Module B (Established)
- <1% in Module C (Current)
- Alert if exceeded

Budget 2: Verification Coverage
- 100% required for Module A
- >95% required for Module B
- >80% required for Module C
- Alert if below threshold

Budget 3: Staleness Limits
- Check K5.7-K5.8 facts every hour
- Check K5.6 facts daily
- Check K5.5 facts monthly
- Purge if stale beyond 2× expected lifetime

Budget 4: Derivation Depth
- Max 10 derivation steps
- Beyond 10 → requires re-verification
- Prevents error accumulation

HOLES:
- Need calibration of thresholds
- Need computational budget for checking
```

Block 7: META-UPDATE LOOP

```
KNOWLEDGE EVOLUTION TRACKING:
```

```
Meta-Law 1: Fact Type Discovery
"If facts consistently fail to map to 343 types, log pattern"
"If pattern repeats, propose new fact type"
"Require approval before adding to taxonomy"

Meta-Law 2: Contradiction Learning
"Log all contradictions + resolution"
"Detect patterns in contradictions"
"Create preventive rules"

Meta-Law 3: Verification Effectiveness
"Track which verification types most reliable"
"Adjust confidence weights"
"Flag unreliable sources"

Meta-Law 4: Domain Expansion
"Detect new entity types not in 9 domains"
"Propose domain additions"
"Require structural validation"

HOLES:
- Need pattern detection system
- Need approval mechanism for taxonomy changes
```

Appendix A: THREAT MODEL

```
THREATS TO KNOWLEDGE INTEGRITY:

T1: Contradiction Injection
- Malicious insertion of conflicting facts
- Accidental assertion of incompatible claims
- Temporal contradictions (old vs new facts)

T2: Verification Failure
- Claimed verification doesn't exist
- Verification invalid but accepted
- Source unreliable

T3: Derivation Corruption
- Invalid inference rules
- Premise errors compound
- Circular derivations

T4: Scope Creep
- Universal claims from limited data
```

```
- Over-generalization
- Context-dependent facts treated as universal

T5: Staleness Accumulation
- Time-dependent facts not updated
- Gradual drift from truth
- Compound staleness (derived from stale)

T6: Domain Confusion
- Entities miscategorized
- Relations applied to wrong domains
- Type errors

T7: Authority Gaming
- Low-authority facts masquerading as high
- Module hierarchy bypassed
- Provenance spoofing

HOLES:
- Need mitigation for each threat
```

Appendix B: FAILURE→RETURN MAP

```
FAILURE → REPAIR MAPPING:

F1: Type Classification Failure
→ R1: Type Classification Correction
   Re-run all 8 detectors
   Assign by majority vote or authority

F2: Contradiction Detected
→ R2: Contradiction Resolution
   Compare authority levels
   Keep higher, flag lower
   OR create context-qualified versions

F3: Verification Missing
→ R3: Verification Restoration
   Search for verification
   Attempt derivation
   Else downgrade or delete

F4: Derivation Invalid
→ R4: Derivation Repair
   Re-derive from current premises
   Update or delete if changed
```

```
F5: Fact Stale
→ R5: Staleness Refresh
   Mark stale
   Fetch update
   Replace or flag unknown

F6: Scope Violation
→ R6: Scope Correction
   Restrict to valid scope
   Prevent over-generalization

F7: Domain Mismatch
→ R7: Domain Reclassification
   Assign correct domain
   Update related facts

F8: Module Authority Violated
→ R8: Authority Enforcement
   Reject lower-authority override
   Escalate for review

HOLES:
- Need implementation of each repair
```

LML: LAW INTEGRATION

LML Candidates:

✅ LAW K.1: FACT TYPE COMPLETENESS

```
EVERY ASSERTED FACT MUST map to exactly one of 343 fact types.

Scope: All knowledge assertions
Enforcement: Type classifier pre-check
Violation: → NSL.K.TYPE_CLASSIFICATION
```

✅ LAW K.2: NON-CONTRADICTION

```
KNOWLEDGE BASE MUST NOT contain logically contradictory facts.

Scope: All stored facts
Enforcement: Contradiction detector
```

```
Violation: → NSL.K.CONTRADICTION_RESOLUTION
```

✅ LAW K.3: VERIFICATION INTEGRITY

```
FACTS MUST have verification matching their K4.x type.

Scope: All facts except K8.0 (axioms)
Enforcement: Verification validator
Violation: → NSL.K.VERIFICATION_RESTORATION
```

✅ LAW K.4: DERIVATION VALIDITY

```
DERIVED FACTS (K8.3-K8.7) MUST follow from valid premises via valid inference.

Scope: All derived facts
Enforcement: Derivation checker
Violation: → NSL.K.DERIVATION_REPAIR
```

✅ LAW K.5: DIMENSIONAL COHERENCE

```
ALL 8 DIMENSIONS of a fact MUST be mutually consistent.

Scope: All facts
Enforcement: Coherence validator
Violation: → NSL.K.COHERENCE_CORRECTION
```

✅ LAW K.6: AUTHORITY HIERARCHY

```
LOWER-AUTHORITY facts CANNOT override higher-authority facts.

Module hierarchy (high→low):
A (Core: definitional, mathematical, eternal)
B (Established: experimental, historical)
C (Current: consensus, statistical)
D (Derived: inferred)
E (Temporary: contextual, momentary)

Scope: All modules
Enforcement: Authority comparator
Violation: → NSL.K.AUTHORITY_ENFORCEMENT
```

✅ LAW K.7: TEMPORAL VALIDITY

```
TIME-DEPENDENT FACTS (K5.6-K5.8) MUST be refreshed within their stability window.

Scope: All time-sensitive facts
Enforcement: Staleness monitor
Violation: → NSL.K.STALENESS_REFRESH
```

REJECTED (Not Laws):

❌ **Fact Usefulness** → Pragmatic, not law

- "Geography facts are useful" = opinion

❌ **Learning Recommendations** → Strategy, not law

- "Learn math facts before physics" = advice

❌ **Source Quality Judgments** → Measurement, not law

- "Wikipedia is 80% reliable" = empirical

NSL: REPAIR MOVES

NSL.K.TYPE_CLASSIFICATION

```
Trigger: Fact cannot be classified OR classified incorrectly
Move: Run 8-dimensional classifier suite
Effect: Every fact gets valid type signature

Algorithm:
1. Extract (subject, relation, object, context)
2. Run all 8 detectors
3. Assign [K1.x, K2.x, ..., K8.x]
4. Validate coherence
5. Store with type
```

NSL.K.CONTRADICTION_RESOLUTION

```
Trigger: Contradiction detected between facts
Move: Resolve via authority hierarchy
Effect: Maintains consistency

Algorithm:
1. Identify contradicting facts F1, F2
```

```
2. Compare modules: A > B > C > D > E
3. If same module:
   a) Compare verification strength
   b) Compare recency (for time-dependent)
   c) Flag for human review if tied
4. Keep higher-authority fact
5. Delete or contextualize lower
6. Log resolution
```

NSL.K.VERIFICATION_RESTORATION

```
Trigger: Fact lacks required verification
Move: Find or derive verification
Effect: All facts properly grounded

Algorithm:
1. Check fact type K4.x
2. Look for matching verification:
   - K4.1 (math) → search for proof
   - K4.3 (experimental) → search for study
   - K4.4 (historical) → search for record
3. If not found, attempt derivation
4. If impossible:
   - Downgrade confidence
   - OR delete if critical
5. Log verification gap
```

NSL.K.DERIVATION_REPAIR

```
Trigger: Derived fact's premises changed or derivation invalid
Move: Re-derive from current state
Effect: Keeps derived facts synchronized

Algorithm:
1. Identify derived fact F
2. Retrieve derivation chain
3. Check if premises still valid
4. Re-run derivation
5. If result changed → update F
6. If derivation fails → delete F
7. Log derivation update
```

NSL.K.STALENESS_REFRESH

```
Trigger: Time-dependent fact exceeded freshness window
Move: Update or mark stale
Effect: Prevents outdated information

Algorithm:
1. Identify stale fact (via K5.x stability class)
2. Attempt to fetch updated value:
   - Query authoritative source
   - Re-measure if observable
   - Re-derive if dependent
3. If update available → replace
4. If unavailable → mark STALE + reduce confidence
5. Log staleness event
```

NSL.K.SCOPE_CORRECTION

```
Trigger: Fact applied beyond valid scope (K6.x)
Move: Restrict to appropriate scope
Effect: Prevents over-generalization

Algorithm:
1. Detect scope violation
2. Identify valid scope from K6.x
3. Restrict fact to that scope
4. Create scope-qualified version
5. Warn if universal claim from limited data
```

NSL.K.COHERENCE_CORRECTION

```
Trigger: Dimensional inconsistency detected
Move: Adjust to nearest coherent state
Effect: Maintains dimensional consistency

Algorithm:
1. Identify inconsistent dimensions
2. Find minimal adjustment to coherent state
3. Prioritize higher-authority dimensions
4. Apply corrections
5. Validate coherence
6. Log correction
```

NSL.K.AUTHORITY_ENFORCEMENT

```
Trigger: Lower-authority fact attempts to override higher
Move: Reject override, enforce hierarchy
Effect: Maintains knowledge base integrity

Algorithm:
1. Compare module levels
2. If lower trying to override higher → REJECT
3. Log violation attempt
4. Optionally escalate for review
5. Keep higher-authority fact
```

DELTA: INTEGRATION RESULTS

LML_ADD:


```
[
  {
    "law_id": "K.1",
    "text": "Fact type completeness required",
    "scope": "All assertions",
    "enforcement": "Type classifier",
    "ledger": "K.1.v1.2026-02-13"
  },
  {
    "law_id": "K.2",
    "text": "Non-contradiction required",
    "scope": "All facts",
    "enforcement": "Contradiction detector",
    "ledger": "K.2.v1.2026-02-13"
  },
  {
    "law_id": "K.3",
    "text": "Verification integrity required",
    "scope": "All non-axiomatic facts",
    "enforcement": "Verification validator",
    "ledger": "K.3.v1.2026-02-13"
  },
  {
    "law_id": "K.4",
    "text": "Derivation validity required",
    "scope": "All derived facts",
    "enforcement": "Derivation checker",
    "ledger": "K.4.v1.2026-02-13"
```

```
  },
  {
    "law_id": "K.5",
    "text": "Dimensional coherence required",
    "scope": "All facts",
    "enforcement": "Coherence validator",
    "ledger": "K.5.v1.2026-02-13"
  },
  {
    "law_id": "K.6",
    "text": "Authority hierarchy enforced",
    "scope": "All modules",
    "enforcement": "Authority comparator",
    "ledger": "K.6.v1.2026-02-13"
  },
  {
    "law_id": "K.7",
    "text": "Temporal validity maintained",
    "scope": "Time-sensitive facts",
    "enforcement": "Staleness monitor",
    "ledger": "K.7.v1.2026-02-13"
  }
]
```


NSL_ADD:


```
[
  {
    "repair_id": "NSL.K.TYPE_CLASSIFICATION",
    "trigger": "Fact unclassified or misclassified",
    "move": "Run 8-dimensional classifier",
    "effect": "Valid type assignment",
    "maps_failures": ["F1"]
  },
  {
    "repair_id": "NSL.K.CONTRADICTION_RESOLUTION",
    "trigger": "Logical contradiction",
    "move": "Authority-based resolution",
    "effect": "Consistency maintained",
    "maps_failures": ["F2"]
  },
  {
    "repair_id": "NSL.K.VERIFICATION_RESTORATION",
    "trigger": "Missing verification",
    "move": "Find or derive verification",
    "effect": "Grounded knowledge",
```

```
    "maps_failures": ["F3"]
  },
  {
    "repair_id": "NSL.K.DERIVATION_REPAIR",
    "trigger": "Invalid derivation",
    "move": "Re-derive from current state",
    "effect": "Synchronized derived facts",
    "maps_failures": ["F4"]
  },
  {
    "repair_id": "NSL.K.STALENESS_REFRESH",
    "trigger": "Expired freshness",
    "move": "Update or mark stale",
    "effect": "Current information",
    "maps_failures": ["F5"]
  },
  {
    "repair_id": "NSL.K.SCOPE_CORRECTION",
    "trigger": "Scope violation",
    "move": "Restrict to valid scope",
    "effect": "Prevents over-generalization",
    "maps_failures": ["F6"]
  },
  {
    "repair_id": "NSL.K.COHERENCE_CORRECTION",
    "trigger": "Dimensional inconsistency",
    "move": "Adjust to coherent state",
    "effect": "Consistent dimensions",
    "maps_failures": ["dimensional mismatch"]
  },
  {
    "repair_id": "NSL.K.AUTHORITY_ENFORCEMENT",
    "trigger": "Authority hierarchy violation",
    "move": "Reject lower-authority override",
    "effect": "Integrity maintained",
    "maps_failures": ["F8"]
  }
]
```

DETECTION_SPEC:

```
- 8 Dimensional Classifiers (K1-K8 detectors)
- Contradiction Detector
- Staleness Monitor
- Verification Validator
- Derivation Checker
```

- Coherence Validator
- Authority Comparator
- Scope Analyzer

BLOCKERS:

HOLES_BLOCKING_LML:


```
[
  {
    "hole_id": "K.H.1",
    "needed_for": "LAW K.2 enforcement",
    "missing": "Fast contradiction detection algorithm",
    "consequence": "Cannot efficiently detect all contradictions"
  },
  {
    "hole_id": "K.H.2",
    "needed_for": "LAW K.3 enforcement",
    "missing": "Verification source authority ranking",
    "consequence": "Cannot determine which verifications are trustworthy"
  }
]
```


HOLES_BLOCKING_NSL:


```
[
  {
    "hole_id": "K.H.3",
    "needed_for": "NSL.K.DERIVATION_REPAIR",
    "missing": "Automated theorem prover / inference engine",
    "consequence": "Cannot automatically re-derive facts"
  },
  {
    "hole_id": "K.H.4",
    "needed_for": "NSL.K.STALENESS_REFRESH",
    "missing": "Authoritative source API connections",
    "consequence": "Cannot automatically fetch updated facts"
  }
]
```

SUMMARY

From EFUE collapse (343 fact types):

✅ **Generated 7 LML Laws:**

- Fact type completeness
- Non-contradiction
- Verification integrity
- Derivation validity
- Dimensional coherence
- Authority hierarchy
- Temporal validity

✅ **Generated 8 NSL Repair Moves:**

- Type classification
- Contradiction resolution
- Verification restoration
- Derivation repair
- Staleness refresh
- Scope correction
- Coherence correction
- Authority enforcement

✅ **Identified 8 Detection Systems:**

- One per fact dimension + cross-cutting detectors

✅ **Flagged 4 Blocking Holes:**

- Fast contradiction checker
- Source authority ranking
- Automated inference
- Update APIs

Status: Laws + Repairs defined, ready for implementation once holes filled.

COMPLETE: EFUE → UIC → UUIS → LML/NSL pipeline executed.

Factual knowledge successfully converted to executable law system.

Infinite facts → 343 types → 7 laws + 8 repairs.